

Detección de Fraude en Transacciones con Tarjeta de Crédito

Un Pipeline de Machine Learning para Clasificación Binaria con Desbalance Extremo de Clases

Juan Jerez

Proyecto Final — Curso de Machine Learning, Talento Tech

Julio 2026

Resumen. Este trabajo desarrolla un pipeline completo de Machine Learning para la detección de fraude en transacciones con tarjeta de crédito, a partir del dataset público *Credit Card Fraud Detection* (Kaggle, ULB), compuesto por 284.807 transacciones de titulares europeos con apenas un 0,17 % de casos fraudulentos. El desafío central no es de escala, sino de **desbalance extremo de clases**, lo que exige un tratamiento cuidadoso tanto en el entrenamiento como en la evaluación de los modelos. Se comparan dos clasificadores — Regresión Logística y Random Forest, ambos con `class_weight="balanced"` — evaluados mediante precisión, recall, F1-score, matriz de confusión, validación cruzada estratificada, curva ROC (AUC) y curva Precision-Recall (Average Precision). El modelo final, Random Forest, alcanza un F1-score de 0,824 y una precisión de 0,971 sobre el conjunto de prueba, identificando el 71,6 % de los fraudes reales con apenas 2 falsos positivos sobre 56.651 transacciones legítimas. El trabajo documenta además por qué el AUC-ROC puede resultar engañoso en este tipo de problemas, y por qué el Average Precision es la métrica más informativa para justificar la selección final del modelo.

Palabras clave: detección de fraude, clasificación desbalanceada, `class_weight`, Random Forest, curva Precision-Recall, scikit-learn.

1 Introducción

El fraude financiero es uno de los principales riesgos operativos del sector bancario y fintech: además de las pérdidas económicas directas, compromete la confianza de los clientes y exige sistemas de detección capaces de operar sobre grandes volúmenes de transacciones en tiempo real. Este proyecto aborda el problema como una tarea de clasificación binaria supervisada, con el objetivo de construir, evaluar y justificar un pipeline de Machine Learning capaz de distinguir transacciones legítimas de transacciones fraudulentas.

El trabajo cubre el flujo completo de un proyecto de ciencia de datos aplicado: exploración inicial, limpieza, análisis de correlaciones, prevención de fuga de información (*data leakage*), entrenamiento de dos modelos de clasificación, evaluación comparativa bajo métricas apropiadas para clases desbalanceadas, validación cruzada estratificada y selección justificada del modelo final.

2 Dataset y Análisis Exploratorio

Se utilizó el dataset público **Credit Card Fraud Detection**, publicado por el Machine Learning Group de la ULB en Kaggle. Contiene transacciones realizadas por titulares de tarjetas europeos durante dos días de septiembre de 2013. Por motivos de confidencialidad, 28 de las 30 variables predictoras (V1–V28) fueron transformadas mediante PCA por los autores del dataset; únicamente **Time** y **Amount** conservan su escala original.

Propiedad	Valor
Transacciones originales	284.807
Registros duplicados eliminados	1.081
Transacciones tras la limpieza	283.726
Transacciones fraudulentas	492 (0,17 %)
Valores faltantes	0
Variables predictoras	30 (Time, Amount, V1–V28)
Período	Sept. 2013 (2 días)

Tabla 1: Resumen del dataset.

El dataset no presenta valores nulos. Sí se identificaron y eliminaron 1.081 filas duplicadas antes de continuar con el análisis, para evitar ruido y sesgos durante el entrenamiento.

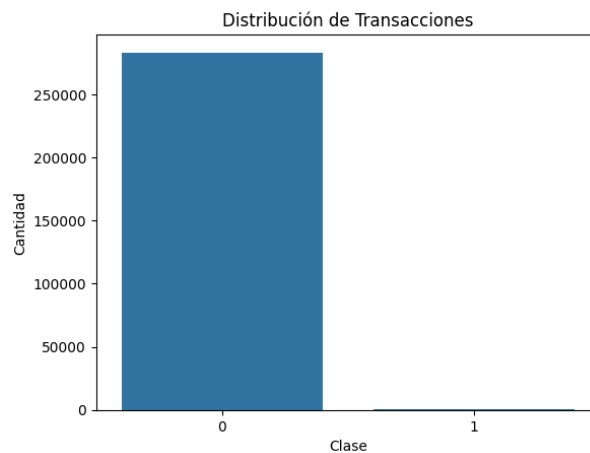


Figura 1: Distribución de la variable objetivo (**Class**). El desbalance es extremo: las transacciones fraudulentas representan apenas el 0,17 % del total.

La Figura 1 confirma el desafío central del proyecto: un clasificador que prediga siempre “no fraude” alcanzaría casi 99,8 % de *accuracy* sin detectar un solo caso real, lo que vuelve a esta métrica prácticamente inútil como criterio de evaluación.

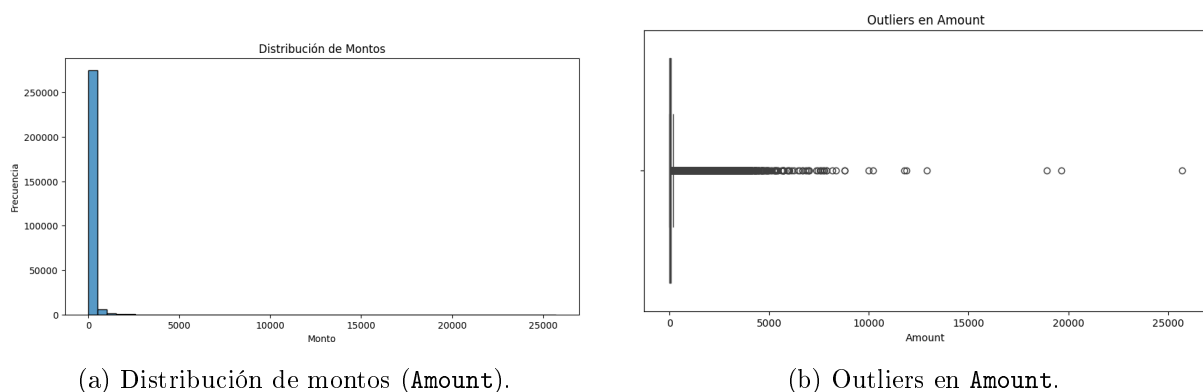


Figura 2: La mayoría de las transacciones presentan montos bajos, aunque existen valores atípicos considerablemente más altos, un patrón habitual — aunque no concluyente por sí solo — en contextos de fraude financiero.

Dado que las variables V1–V28 están anonimizadas mediante PCA, estas correlaciones son útiles para orientar el análisis pero no permiten una interpretación de negocio directa (no es posible

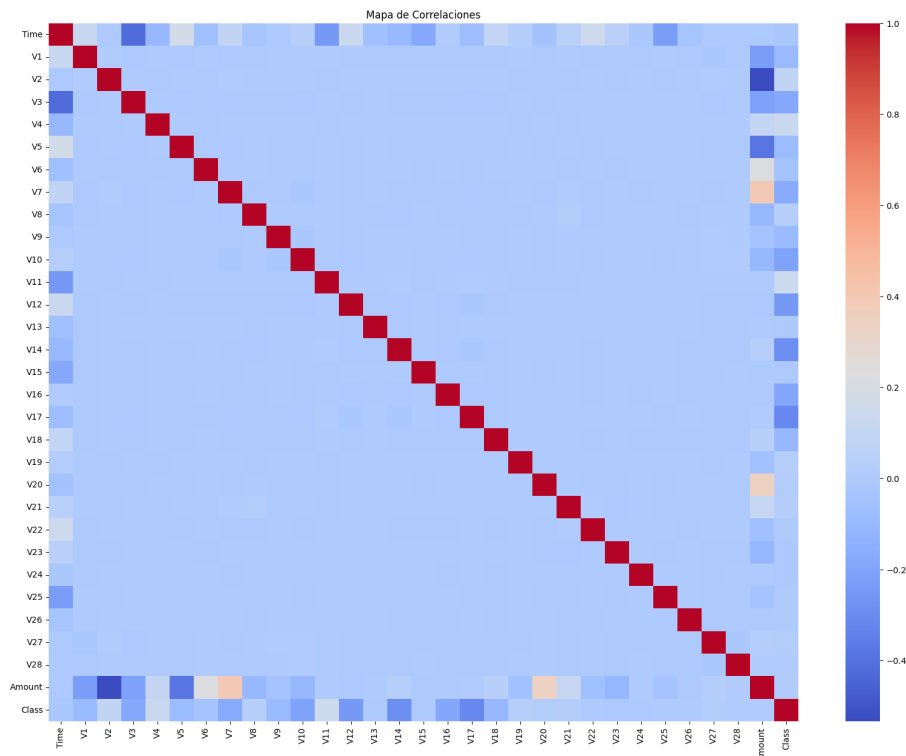


Figura 3: Mapa de correlaciones entre variables. Las variables V17, V14, V12 y V10 muestran la correlación negativa más fuerte con **Class**, mientras que V11, V4 y V2 muestran la correlación positiva más fuerte.

saber, por ejemplo, si V14 está relacionada con el monto, el comercio o el horario de la operación).

3 Metodología

3.1 División del dataset y prevención de fuga de información

El dataset se dividió en 80 % entrenamiento (226.980 transacciones) y 20 % prueba (56.746 transacciones), utilizando `stratify=y` para preservar la proporción de fraude en ambos subconjuntos (0,167 % en ambos casos). El escalado de **Amount** y **Time** mediante `StandardScaler` se ajustó **únicamente** sobre el conjunto de entrenamiento y luego se aplicó al conjunto de prueba, evitando que este último aporte información durante la preparación de los datos.

3.2 Manejo del desbalance de clases

Con apenas 0,17 % de casos positivos, dos decisiones metodológicas fueron centrales:

1. `class_weight="balanced"` en ambos modelos, para que el algoritmo penalice más los errores cometidos sobre la clase minoritaria durante el entrenamiento.
2. **Abandonar la *accuracy* como métrica de referencia**, priorizando en su lugar precisión, recall, F1-score, matriz de confusión, AUC-ROC y, especialmente, Average Precision (AP) sobre la curva Precision-Recall.

4 Modelos de Clasificación

Se entrenaron y compararon dos modelos:

- **Regresión Logística** (`max_iter=1000`, `class_weight="balanced"`): modelo lineal, rápido y directamente interpretable a nivel de coeficientes por variable.
- **Random Forest** (`n_estimators=200`, `class_weight="balanced"`): ensamble de árboles de decisión, capaz de capturar relaciones no lineales entre las variables anonimizadas, a costa de una interpretabilidad menor.

5 Resultados

5.1 Métricas sobre el conjunto de prueba

Modelo	Accuracy	Precision	Recall	F1-score	AUC-ROC	AP
Regresión Logística	0,975	0,056	0,874	0,106	0,966	0,672
Random Forest	0,999	0,971	0,716	0,824	0,945	0,809

Tabla 2: Comparación de métricas entre modelos sobre el conjunto de prueba (56.746 transacciones, 95 fraudes reales).

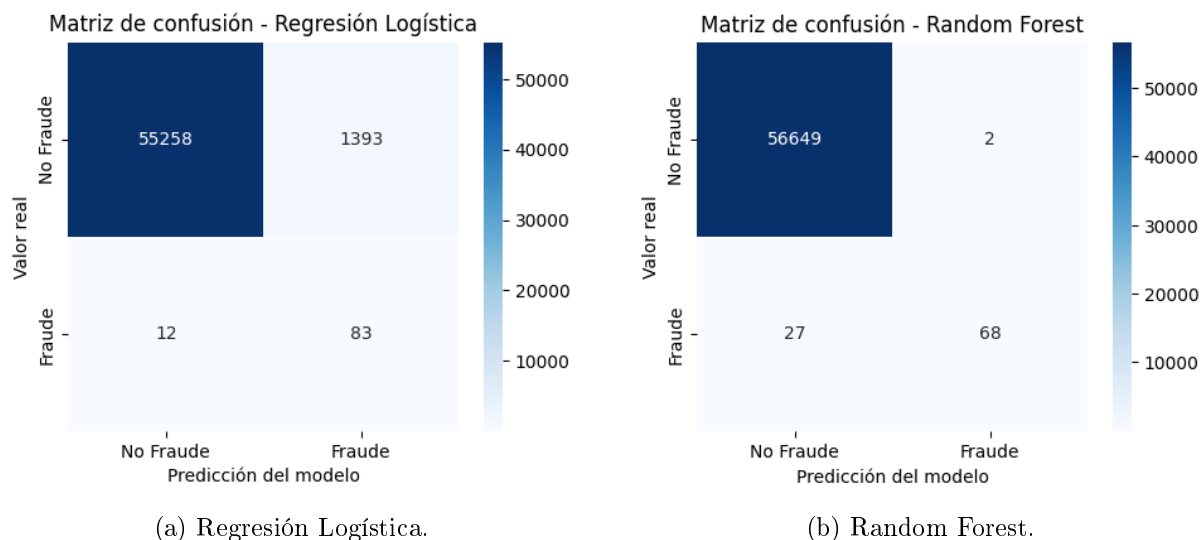


Figura 4: Matrices de confusión sobre el conjunto de prueba. La Regresión Logística detecta más fraudes (83/95) pero genera 1.393 falsos positivos; Random Forest detecta 68/95 fraudes con apenas 2 falsos positivos.

El trade-off central del proyecto queda claramente expuesto en la Figura 5: la Regresión Logística deja pasar solo 12 fraudes reales sin detectar, pero a costa de 1.393 falsas alarmas sobre transacciones legítimas — más de 16 alertas erróneas por cada fraude real encontrado. Random Forest, en cambio, deja 27 fraudes sin detectar, pero genera apenas 2 falsos positivos sobre 56.651 transacciones normales.

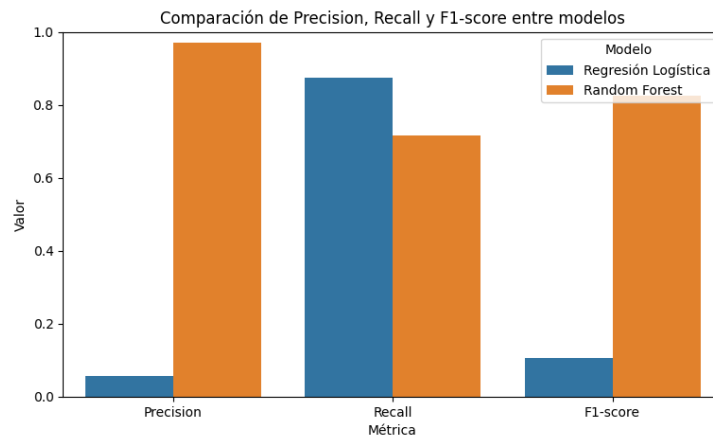


Figura 5: Comparación de precisión, recall y F1-score entre ambos modelos. La *accuracy* se excluye deliberadamente del gráfico por no ser informativa en este contexto de desbalance extremo.

5.2 Validación cruzada

Se aplicó `StratifiedKfold` con 5 particiones, utilizando F1-score como métrica de evaluación en cada fold.

Modelo	F1 promedio (5 folds)	Desvío estándar
Regresión Logística	0,114	0,013
Random Forest	0,837	0,046

Tabla 3: Validación cruzada estratificada (*scoring="f1"*). Incluso el peor fold de Random Forest (0,756) supera ampliamente al mejor fold de la Regresión Logística (0,135).

5.3 Curva ROC y curva Precision-Recall

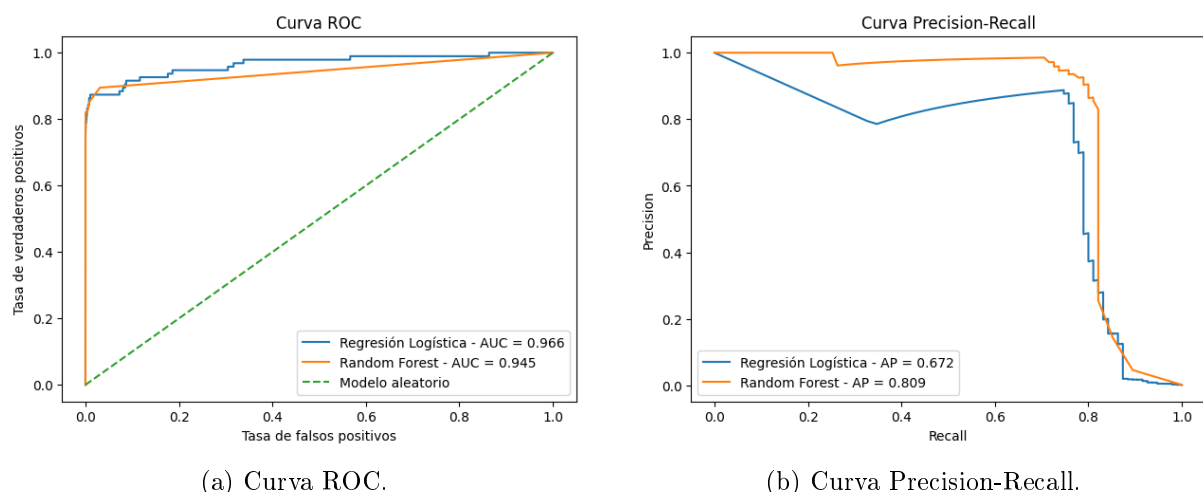


Figura 6: Comparación de curvas ROC y Precision-Recall. El orden de los modelos se invierte entre ambas métricas: la Regresión Logística tiene mayor AUC-ROC (0,966 vs. 0,945), pero Random Forest tiene mayor Average Precision (0,809 vs. 0,672).

Este cruce de resultados es, en sí mismo, uno de los hallazgos más relevantes del proyecto: en datasets con un desbalance tan extremo, el AUC-ROC puede resultar engañosamente optimista,

ya que la enorme cantidad de transacciones negativas hace que incluso un modelo con muchos falsos positivos mantenga una tasa de falsos positivos *relativa* baja. El Average Precision, al centrarse exclusivamente en la clase minoritaria, refleja con mayor fidelidad el costo real de cada modelo en términos de falsas alarmas.

6 Selección del Modelo Final

Random Forest fue seleccionado como modelo final. Los tres criterios cuantitativos considerados — F1-score (0,824 vs. 0,106), Average Precision (0,809 vs. 0,672) y validación cruzada (F1 promedio 0,837 vs. 0,114) — coinciden en favorecerlo de forma consistente, a pesar de que la Regresión Logística obtiene mayor recall (0,874 vs. 0,716) y mayor AUC-ROC (0,966 vs. 0,945).

La razón operativa es directa: con una precisión de apenas 5,6 %, la Regresión Logística generaría en la práctica un volumen de alertas imposible de operar para un equipo de revisión de fraude. Random Forest, con 97,1 % de precisión, permite construir un sistema de alertas manejable, donde casi todas las transacciones marcadas son efectivamente fraudulentas.

7 Limitaciones

- Las variables V1–V28 están anonimizadas mediante PCA, lo que impide interpretar su significado de negocio y limita la explicabilidad del modelo final.
- Con solo 492 fraudes en todo el dataset, cualquier evaluación — en especial la validación cruzada — puede ser sensible a cómo se distribuyen esos pocos casos positivos entre folds.
- El modelo final deja 27 fraudes sin detectar sobre el conjunto de prueba; reducir ese número sería una prioridad de negocio en un contexto real.
- Se utilizó `class_weight="balanced"` como única estrategia de balanceo; no se probaron técnicas alternativas como SMOTE, submuestreo o ajuste manual del umbral de decisión.
- No se realizó una búsqueda de hiperparámetros (`GridSearchCV` / `RandomizedSearchCV`); los modelos se entrenaron con configuraciones razonables elegidas manualmente.
- El dataset corresponde a solo dos días de septiembre de 2013 de tarjetahabientes europeos. Un sistema en producción necesitaría reentrenarse periódicamente, dado que los patrones de fraude evolucionan con el tiempo.

8 Aplicación Práctica

Un modelo como el desarrollado podría integrarse como una capa de detección automática dentro de un sistema de procesamiento de pagos: las transacciones con mayor probabilidad de fraude según `predict_proba` se enviarían a revisión manual o bloqueo temporal, mientras que el resto se procesa con normalidad. Con 97,1 % de precisión, un equipo de revisión podría operar sobre las alertas del modelo sin verse desbordado por falsos positivos — algo inviable con la Regresión Logística en su configuración actual.

Este notebook tiene fines educativos. Antes de un despliegue real sería necesario validar el modelo con datos más recientes, monitorear su comportamiento en producción, ajustar el umbral de decisión según el costo real de cada tipo de error para el negocio, y contemplar los aspectos éticos y regulatorios propios del sector financiero.

9 Conclusión

Este proyecto desarrolla un flujo completo de Machine Learning aplicado a la detección de fraude en tarjetas de crédito, con especial atención a los dos desafíos que definen este tipo de problemas: el desbalance extremo de clases y la elección de métricas de evaluación adecuadas. La comparación entre Regresión Logística y Random Forest muestra que el modelo con mejor *ranking* teórico (mayor AUC-ROC) no es necesariamente el más útil en la práctica, y que el Average Precision es un criterio más confiable para seleccionar modelos en contextos de fraude. Random Forest, con un F1-score de 0,824 y una precisión de 97,1 %, resulta el modelo final más adecuado para un sistema de alertas operativamente viable.

Dataset: Machine Learning Group – ULB, *Credit Card Fraud Detection*, Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Herramientas: Python, pandas, scikit-learn (LogisticRegression, RandomForestClassifier, StratifiedKFold, class_weight, precision_recall_curve), matplotlib, seaborn, Google Colab.